BICOL UNIVERSITY - POLANGUI
Web Systems and Technology

ALSOR CO.

BS INFORMATION TECHNOLOGY

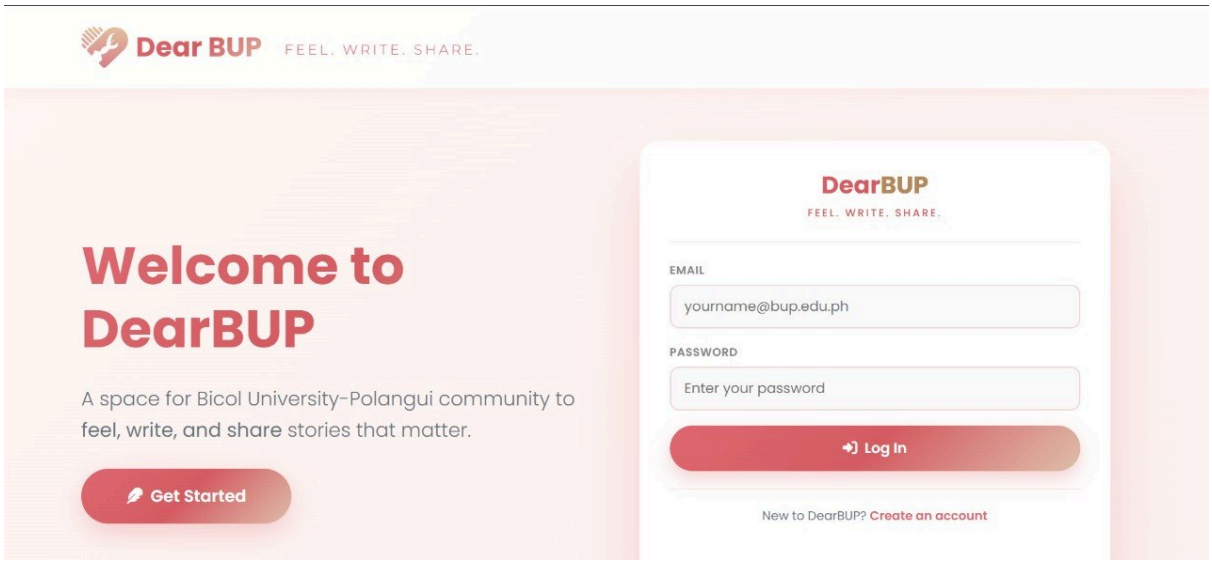
PROPONENTS:

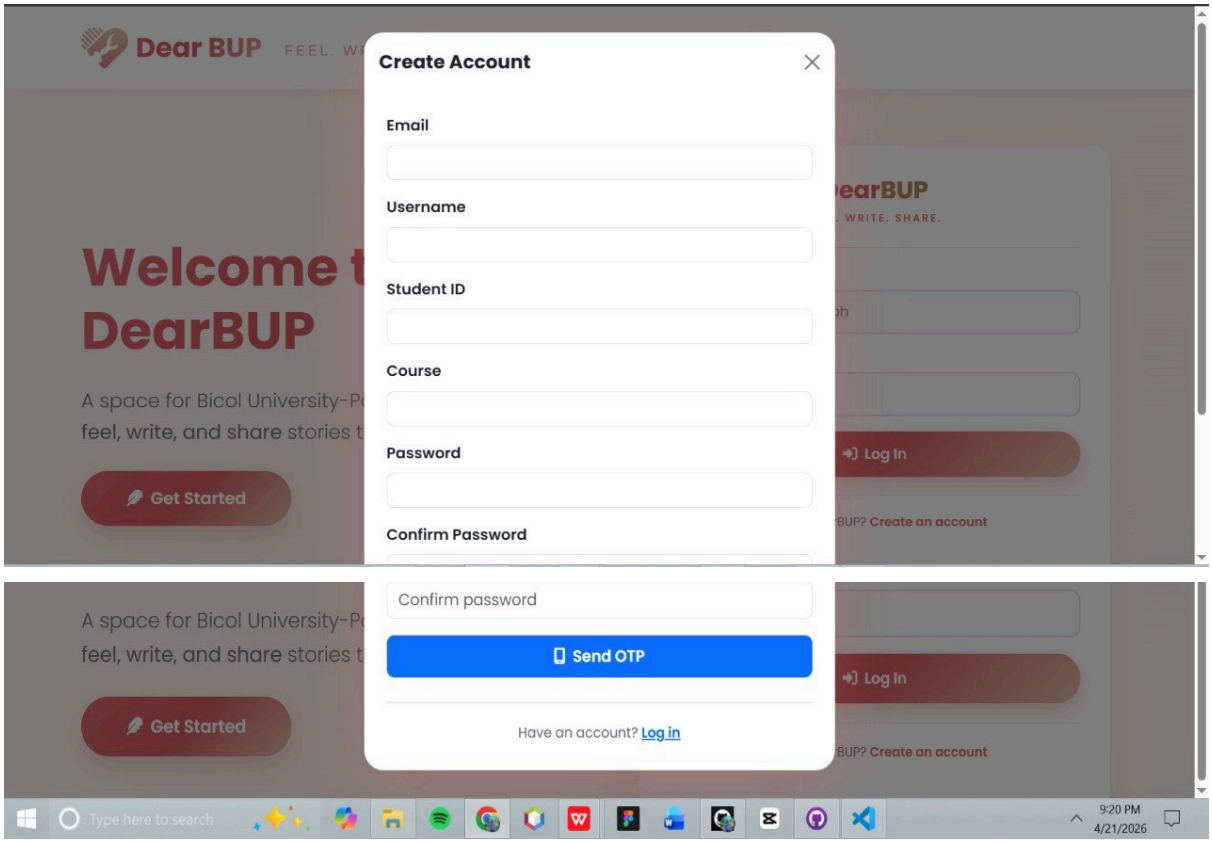
Ivan Jacob Milan
Nadel Volante
James Espinosa
Erick John Bonaobra
Manuel Angelo Loma

DearBUP
Feel. Write. Share.

Week 13: Phase 4 Form Submission and Data Insertion

FORMS INTERFACE





If the new user came across our website, they must register first in the form using their university email. The email they provide must end with the university domain which is *bicol-u.edu.ph* or else they will not be able to create an account. The validated email will receive an OTP and will now proceed to its profile dashboard.

API CONNECTIONS SCRIPT

- Database Connection

```
import mongoose from "mongoose";

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI, {
      serverSelectionTimeoutMS: 5000,
    });

    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error("MongoDB Connection Error:", error.message);
    process.exit(1); // Stop server if database fails
  }
};

export default connectDB;
```

- Sending OTP through email

```
import nodemailer from "nodemailer";

// create transporter
const transporter = nodemailer.createTransport({
  service: "gmail",
  auth: {
    user: process.env.GMAIL_USER,
    pass: process.env.GMAIL_PASS,
  },
});
// send OTP email
export const sendOtpEmail = async (email, code) => {
  try {
    await transporter.sendMail({
      from: `"DearBUP" <${process.env.GMAIL_USER}>`,
      to: email,
      subject: "DearBUP Verification Code",
      html: `
        <div style="font-family: Arial; text-align: center;">
          <h2>Your OTP Code</h2>
          <h1 style="letter-spacing: 5px;">${code}</h1>
          <p>This code will expire in 5 minutes.</p>
        </div>
```

```
      `,
    });

    console.log("✅ Email sent to:", email);

  } catch (error) {
    console.error("❌ Email error:", error.message);
    throw new Error("Failed to send email");
  }
};
```

- Spotify Connection

```
import axios from "axios";

let token = null;
let tokenExpires = null;

export const getSpotifyToken = async () => {
  if (token && tokenExpires > Date.now()) return token;

  const response = await axios.post(
    "https://accounts.spotify.com/api/token",
    "grant_type=client_credentials",
    {
      headers: {
        Authorization:
          "Basic " +
          Buffer.from(
            process.env.SPOTIFY_CLIENT_ID + ":" +
process.env.SPOTIFY_CLIENT_SECRET
          ).toString("base64"),
        "Content-Type": "application/x-www-form-urlencoded",
      },
    }
  );

  token = response.data.access_token;
  tokenExpires = Date.now() + response.data.expires_in * 1000;

  return token;
};
```

- Entry Point

```
import "dotenv/config";
import express from "express";
import cors from "cors";
import connectDB from "./config/db.js";
import authRoutes from "./routes/authRoutes.js";
import postRoutes from "./routes/postRoutes.js";
import letterRoutes from "./routes/letterRoutes.js";
import feedbackRoutes from "./routes/feedbackRoutes.js";

// Initialize Database
connectDB();


const app = express();


// Middleware
app.use(cors({
  origin: "*",
  methods: ["GET", "POST", "PUT", "DELETE"],
  credentials: false
}));
app.use(express.json());
app.use("/api/auth", authRoutes);
app.use("/api/posts", postRoutes);
app.use("/api/letters", letterRoutes);
app.use("/api/feedback", feedbackRoutes);


// Health Check
app.get("/", (req, res) => {
  res.send("Backend is running...");
});


// Port Configuration
const PORT = process.env.PORT || 5000;

// Use "0.0.0.0" for deployment readiness, but console log localhost for dev
app.listen(PORT, "0.0.0.0", () => {
  console.log(`🚀 Server running on http://localhost:${PORT}`);
  console.log(`🎵 Spotify Login:
http://localhost:${PORT}/api/spotify/login`);
});
```

MONGODB DOCUMENT INSERTED

- Posts Schema

```
import mongoose from "mongoose";

// ---------------- COMMENT ----------------
const CommentSchema = new mongoose.Schema({
  user_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  text: {
    type: String,
    required: true,
  },
  date: {
    type: Date,
    default: Date.now,
  },
});

// ---------------- POST ----------------
const PostSchema = new mongoose.Schema(
  {
    user_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },

    content: {
      type: String,
      required: true,
    },

    design_template: {
      type: String,
      default: null,
    },

    spotify_track_url: {
      type: String,
      default: null,
    },

    is_anonymous: {
      type: Boolean,
      default: false,
    },

    // ⭐ ORGANIZATION FEED SUPPORT
    org_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "CBO",
      default: null,
    },
```

- Posts Data on MongoDB

```
_id: ObjectId('69e62a7f26a4844e36979557')
user_id : ObjectId('69e5159359f60dfc4ebb089d')
content : "Hello there"
design_template : null
spotify_track_url : null
is_anonymous : false
reactions : Array (1)
shared_from : null
comments : Array (empty)
createdAt : 2026-04-20T13:30:39.754+00:00
updatedAt : 2026-04-20T13:30:42.041+00:00
__v : 1
```

```
_id: ObjectId('69e668e16fd9bb259e568eec')
user_id : ObjectId('69e5159359f60dfc4ebb089d')
content : "awdawda"
design_template : null
spotify_track_url : null
is_anonymous : false
reactions : Array (empty)
shared_from : null
comments : Array (empty)
createdAt : 2026-04-20T17:56:49.518+00:00
updatedAt : 2026-04-20T17:56:49.518+00:00
__v : 0
```

```
_id: ObjectId('69e669ea6fd9bb259e568ef5')
user_id : ObjectId('69e5159359f60dfc4ebb089d')
content : "sfsfsefs"
design_template : null
spotify_track_url : null
is_anonymous : false
reactions : Array (1)
shared_from : null
```

- Letter schema

final-project-Alsor-co-BSIT2a > backend > src > models > LetterModel.js > default

```javascript
import mongoose from "mongoose";

const LetterSchema = new mongoose.Schema({
  user_id: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  letter_title: { type: String, required: true },
  letter_content: { type: String, required: true },
  letter_type: { type: String, enum: ["confession", "gratitude", "concern", "general"] },
  display_name: { type: String },
  is_anonymous: { type: Boolean, default: false },

  // --- NEW SPOTIFY FIELDS ---
  spotifySongName: { type: String },
  spotifyArtist: { type: String },
  spotifyTrackUri: { type: String },
  spotifyImageUrl: { type: String },
  // -------------------------

}, { timestamps: true });

export default mongoose.model("Letter", LetterSchema);
```

- Letter data on MongoDB

```
_id: ObjectId('69e633cf26a4844e369795c1')
user_id : ObjectId('69e5159359f60dfc4ebb089d')
letter_title : "to lovey"
letter_content : "dear lovey, mahal pa rin kita."
letter_type : "gratitude"
display_name : "Anonymous"
is_anonymous : true
spotifySongName : "The One You Love"
spotifyArtist : "Glenn Frey"
spotifyTrackUri : "spotify:track:4uvARnZIajUDXWXPifhK3B"
spotifyImageUrl : "https://i.scdn.co/image/ab67616d0000b2736281626dc98c227bf8dfc34d"
createdAt : 2026-04-20T14:10:23.796+00:00
updatedAt : 2026-04-20T14:10:23.796+00:00
__v : 0
```

```
_id: ObjectId('69e6375526a4844e369795e0')
user_id : ObjectId('69e5159359f60dfc4ebb089d')
letter_title : "Pride"
letter_content : "Pride ciken"
letter_type : "general"
display_name : "angelotzy"
is_anonymous : false
spotifySongName : "Chicken Banana"
spotifyArtist : "Crazy Music Channel"
spotifyTrackUri : "spotify:track:2DW0Mowto3hrXkFBQt0nye"
spotifyImageUrl : "https://i.scdn.co/image/ab67616d0000b2731c786b539e61217e5bdbde16"
createdAt : 2026-04-20T14:25:25.100+00:00
updatedAt : 2026-04-20T14:25:25.100+00:00
__v : 0
```

```
_id: ObjectId('69e64a46e0d8c5f42f45a3bf')
user_id : ObjectId('69e5159359f60dfc4ebb089d')
letter_title : "awda"
letter_content : "awdawd"
```

- Notification Schema

```
import mongoose from "mongoose";

const NotificationSchema = new mongoose.Schema(
  {
    user_id: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true, // receiver
    },

    from_user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      default: null,
    },

    type: {
      type: String,
      enum: [
        "reaction",
        "comment",
        "share",
        "event",
        "organization",
      ],
      required: true,
    },

    message: {
      type: String,
      required: true,
    },

    related_id: {
      type: mongoose.Schema.Types.ObjectId,
      default: null,
    },

    is_read: {
      type: Boolean,
      default: false,
    },
  },
  { timestamps: true }
);

export default mongoose.model("Notification", NotificationSchema);
```

- Notification data on MongoDB

ADD DATA | UPDATE | DELETE | EXPORT CODE

```
_id: ObjectId('69eb07e998949df679a921c6')
user_id : ObjectId('69e5159359f60dfc4ebb089d')
from_user : ObjectId('69de438e73951e3356341863')
type : "reaction"
message : "reacted to your post"
related_id : ObjectId('69ea19627e53528129339b70')
is_read : true
createdAt : 2026-04-24T06:04:25.231+00:00
updatedAt : 2026-04-24T06:07:14.810+00:00
__v : 0
```

```
_id: ObjectId('69eb086798949df679a921d7')
user_id : ObjectId('69de438e73951e3356341863')
from_user : ObjectId('69e5159359f60dfc4ebb089d')
type : "reaction"
message : "reacted to your post"
related_id : ObjectId('69e73c10b08007b99970b72f')
is_read : true
createdAt : 2026-04-24T06:06:31.508+00:00
updatedAt : 2026-04-24T06:06:42.602+00:00
__v : 0
```

```
_id: ObjectId('69eb08a998949df679a921fa')
user_id : ObjectId('69e5159359f60dfc4ebb089d')
from_user : ObjectId('69de438e73951e3356341863')
type : "share"
message : "shared your post"
related_id : ObjectId('69ea19627e53528129339b70')
is_read : true
createdAt : 2026-04-24T06:07:37.991+00:00
updatedAt : 2026-04-24T06:07:48.034+00:00
__v : 0
```

- Users Schema

```
import mongoose from "mongoose";

const courseMap = {
  BSIT: "Bachelor of Science in Information Technology",
  BSCS: "Bachelor of Science in Computer Science",
  BSIS: "Bachelor of Science in Information System",
  "BSIT-ANIM": "Bachelor of Science in Information Technology Major in Animation"
};

const UserSchema = new mongoose.Schema(
  {
    email: { type: String, required: true, unique: true },
    student_id: { type: String, required: true, unique: true },
    username: { type: String, required: true, unique: true },

    course: {
      type: String,
      enum: Object.values(courseMap),
      required: true,
      set: (value) => {
        const normalized = value?.trim().toUpperCase();
        return courseMap[normalized] || value;
      }
    },

    department: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Department"
    },

    organization: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "CBO",
      required: false
    },

    password: { type: String, required: true },

    user_type: {
      type: String,
      enum: ["student", "officer", "department_head", "admin"],
      required: true
    },

    display_name: { type: String },

    avatar_url: { type: String, default: "" },
    bio: { type: String, default: "" },

    is_verified: { type: Boolean, default: false }
  },
  { timestamps: true }
);

export default mongoose.model("User", UserSchema);
```

- Users data on MongoDB


```
_id: ObjectId('69d756293dfce9a3af7a8bc9')
email : "nadel@gmail.com"
username : "nadel"
password : "$2b$10$hPh1G4oOu6OPKCe2BBtWkO8ErOxqOPmidgN.uSxuPbVGdDUKqDd/a"
user_type : "student"
display_name : "nadel"
createdAt : 2026-04-09T07:32:57.841+00:00
updatedAt : 2026-04-09T07:32:57.841+00:00
__v : 0
```

```
_id: ObjectId('69dbd49bf4cbd04f7690cef6')
email : "milanivanjacob@gmail.com"
student_id : "2024-0105829"
username : "Ivan Jacob Milan"
course : "Information Technology"
password : "$2b$10$bTOWxmhnZdjgFzj/xVAZSecre2VCe8iJfAnql5h675VP1ahTJcl1u"
user_type : "student"
display_name : "Test Account"
avatar_url : ""
bio : ""
is_verified : false
createdAt : 2026-04-12T17:21:31.814+00:00
updatedAt : 2026-04-12T17:21:31.814+00:00
__v : 0
```

```
_id: ObjectId('69dc42be2ff31565f017a1b2')
email : "ivanjacobgojar.milan24@bicol-u.edu.ph"
student_id : "20240001"
username : "ivan"
course : "Information Technology"
password : "$2b$10$HHOVjc1yvoXXqry0Bnsg1uIr4UG6ko34TWPiTzrzGWblKrS4aadx."
user_type : "student"
display_name : "ivan"
avatar_url : ""
```

GITHUB COMMITS (Frontend)

codewithloma / final-project-Alsor-co-BSIT2a

Code | Issues | Pull requests | Actions | Projects | Wiki | Security and quality | More

Commits

History for final-project-Alsor-co-BSIT2a / frontend on main

All users | All time

Commits on Apr 21, 2026

- **Updated Log In Form** — rckjhnb committed 14 hours ago — 5b361b6
- **fixed the logout and Post Reactions** — chivarly committed 18 hours ago — ffb69c1
- **Ceonnected Frontend to Backend | Refactored and Modularized scripts** — chivarly committed yesterday — bac1bd8

Commits on Apr 20, 2026

- **added vite and refactor project structure** — chivarly committed 2 days ago — 6bd097e

Commits on Apr 19, 2026

- **Added Organization Page** — rckjhnb committed 3 days ago — f22512e

Commits on Apr 18, 2026

- **Completed main page** — rckjhnb committed 4 days ago — 0dcfea8
- **Completed main page** — rckjhnb committed 4 days ago — 82a2ace
- **Completed LogIn/Register Form** — rckjhnb committed 4 days ago — 9b83016

Commits on Mar 26, 2026

- **Add project folder structure** — codewithloma committed last month — ad942b4

End of commit history for this file

Terms | Privacy | Security | Status | Community | Docs | Contact | Manage cookies
Do not share my personal information

© 2026 GitHub, Inc.

GITHUB COMMITS (Backend)

codewithloma / final-project-Alsor-co-BSIT2a

Code | Issues | Pull requests | Actions | Projects | Wiki | Security and quality | More

Commits

History for final-project-Alsor-co-BSIT2a / backend on `main`

All users | All time

Commits on Apr 21, 2026

Fixed postModel
chivarly committed yesterday
e787401

Commits on Apr 14, 2026

completed core backend features (Auth with OTP, Letters, Posts, and Dept Feedback)
chivarly committed last week
c0e1a4f

Commits on Apr 10, 2026

feat: enhance auth flow, secure post actions, and add JWT middleware
chivarly committed 2 weeks ago
f97649b

Add .gitignore to ignore environment files and node_modules
chivarly committed 2 weeks ago
5377c80

Commits on Apr 9, 2026

partial backend
chivarly committed 2 weeks ago
9b3a4ad

Commits on Mar 26, 2026

Add project folder structure
codewithloma committed last month
ad942b4

End of commit history for this file

Terms | Privacy | Security | Status | Community | Docs | Contact | Manage cookies
Do not share my personal information

© 2026 GitHub, Inc.

REFLECTIONS

James P. Espinosa | Project Manager | Document Officer

For today's activity, I learned that before you can actually set your website you need to have an APi connection. My contribution in this activity is the compilation of documentation or screenshots of the requirements for this Lab report. I prepare a note for the explanation of the flow on our forms or the register and log-in phase.

Ivan Jacob Milan | Backend Developer

During this development process, I improved the organization management system by implementing role-based access control, ensuring that only the organization president can create an organization while other officer permissions are validated securely. I also enhanced the backend logic by preventing duplicate member approvals, protecting organization actions from unauthorized users, and preparing the notification system to support future tracking of post interactions and organization activities.

One of the most valuable things I learned was how to properly connect the frontend to the backend so that user actions in the interface can communicate with the database through API routes and controllers. This helped me understand how data flows between the client side and server side, making the application more dynamic, secure, and scalable for future features.

Erick John Bonaobra | Frontend Develope

On this week, I updated the Landing or Log In page, where the users can now see the Log In form when they open the website. With the help of the Backend Developer, I successfully connected it to our backend. This helped me realize that cooperation and collaboration is important in making a website.

Manuel Angelo Loma | GitHub Manager

This week was a big learning experience for me. I was in charge of organizing our repository and making sure everything was updated. I focused on arranging the folders properly and checking all the files that were uploaded. I also tried to keep everything clear and organized so our group wouldn't get confused or have problems like merge conflicts later on.

Nadel B. Volante | Database Manager

This week, I was tasked with checking whether the data that was submitted in our website was successfully updated into or MongoDB cluster. I also checked whether the data followed the schema provided in our backend code. This week made me realize that database connectivity is very important in web development since it covers all the real-time information that is being added to a website.